

Boş / Eksik Verilerin Yönetimi ve Doldurma (Imputation)

Dersin Amaçları

- Eksik verinin ne olduğunu anlamak: neden oluşur, nasıl tespit edilir.
- Eksik veri nedeniyle yaşanabilecek sorunları (analiz hatası, model çökmesi, bias, veri kaybı) kavramak.
- Farklı doldurma stratejilerini tanımak: basit (ortalama, medyan...), ileri yöntemler (KNN, regresyon, çoklu imputasyon...).
- Python / pandas / scikit-learn ile pratik örnekler yaparak uygulamalı öğrenmek.
- Hangi duruma hangi stratejinin uygun olduğuna karar verebilmek (örneğin dağılım, missing oranın azlığı/çokluğu, veri tipi vs'ye göre).

Eksik Veri Nedir & Neden Önemli?

- Eksik veri (missing / null / NaN): Bir gözlem (satır) ve özellik (sütun) için geçerli bir değer olmaması demektir. Bu "0" değerinden farklıdır.
- Eksik veri nedenleri: ölçüm hatası, veri girişi hatası, sensör arızası, kullanıcı cevapsızlığı (anket), kayıt hatası, veri kaybı, dataset birleşimi vb.
- Eksik verilerin etkileri: analiz döneminde ortalamalar, dağılımlar, korelasyonlar bozulabilir; bazı modeller NaN ile çalışamaz; veri kaybı olabilir. Bu yüzden missing data ile çalışırken dikkatli olmak gerekir.
- Eksik veri ile baş etme temelde 3 yol vardır:
 - Satır / sütun silme (drop).
 - Eksik değerleri doldurma (imputation).
 - Eksikliği model / analiz metodunun göze alamayacağı durumlarda özel yöntemler (model-based, generative, maximum-likelihood, multiple imputation ...).

Yaygın Eksik Veri Doldurma / Imputation Yöntemleri

Yöntem/Strateji	Ne zaman / nerede kullanılır / Avantaj / Dezavantaj
Ortalama (Mean), Medyan, Mod	Sayısal değişkenlerde; düşük oranda eksik varsa; veri dağılımı çok bozulmamışsa. Basit ve hızlıdır. Ancak dağılım bozursa (çarpık, çok uç değer) ortalama yanıltıcı olabilir. Medyan/mod bu durumda daha güvenli olabilir.
Sabit Değer / Sabit Kod (Constant Imputation)	Eksik veriler için anlamlı sabit bir değer varsa ya da "bilinmiyor" gibi bir değer mantıklı ise (örneğin yaş bilinmiyorsa -1 atamak gibi). Kolay ama dikkatli kullanılmalı.
Forward / Backward Fill (ffill / bfill)	Zaman serisi ya da sıralı veri varsa; önceki ya da sonraki gözleme göre doldurmak mantıklı ise. Ancak sıranın mantıklı olması gerekir.
KNN Imputation (En yakın komşulara göre doldurma)	Eksik değer başka değişkenlerle ilişkiliyse ve benzer gözlemler varsa; özellikle çok değişkenli veri setlerinde.

Yöntem/Strateji	Ne zaman / nerede kullanılır / Avantaj / Dezavantaj
	Eksik değer bağımsız değilse mantıklı.
Regression / Model-based Imputation	Eksik değişkenin diğer değişkenlerle güçlü ilişkisi varsa; regresyon, karar ağacı vb modellerle tahmin edilebilir. Ancak model varsayımlarına dikkat.
Çoklu İmputation (Multiple Imputation), Iterative Imputer, EM gibi yöntemler	Eksik veri oranı yüksekse, missing değerlerin rastgele dağılmadığı biliniyorsa; sadece bir doldurma yerine çoklu doldurmalar + varyans dahil edilmek isteniyorsa.

```
In [1]: # Örnek veri seti (simülasyon + boş değer oluşturma)

import pandas as pd
import numpy as np

np.random.seed(42)

# 100 satır, 4 sütun
df = pd.DataFrame({
    'age': np.random.randint(18, 70, size=100),
    'income': np.random.normal(50000, 15000, size=100),
    'education_years': np.random.randint(8, 20, size=100),
    'city': np.random.choice(['A', 'B', 'C'], size=100)
})

# Rasgele %10 eksik değer atama
for col in ['income', 'education_years', 'city']:
    df.loc[df.sample(frac=0.1).index, col] = np.nan

df.head()
df.isnull().mean() * 100 # her bir sütunda eksik oranı %
```

```
Out[1]: age          0.0
income        10.0
education_years 10.0
city          10.0
dtype: float64
```

```
In [2]: # Temel Imputation: Mean / Median / Mode

# SimpleImputer kullanarak eksik sayısal değerleri sütunun medyanıyla,
# kategorik verileri ise en sık görülen (mod) ile dolduruyor.

from sklearn.impute import SimpleImputer

# Sayısal değişkenler için medyan, kategorik için mod
num_cols = ['age', 'income', 'education_years']
cat_cols = ['city']

imp_num = SimpleImputer(strategy='median')
df[num_cols] = imp_num.fit_transform(df[num_cols])

imp_cat = SimpleImputer(strategy='most_frequent')
df[cat_cols] = imp_cat.fit_transform(df[cat_cols])

df.isnull().sum()
```

```
Out[2]: age          0
income        0
education_years 0
city          0
dtype: int64
```

```
In [3]: # KNN Imputation

# KNN tabanlı doldurma: eksik değeri olan satırın, diğer satırlara göre benzer (k
# satırlarının ortalaması / medyanı ile doldurma. Özellikle çok değişkenli ilişki

from sklearn.impute import KNNImputer

df2 = df.copy()

imputer = KNNImputer(n_neighbors=5)
df2[num_cols] = imputer.fit_transform(df2[num_cols])

df2.head()
```

```
Out[3]:
```

	age	income	education_years	city
0	56.0	21864.848417	17.0	C
1	69.0	29498.267917	10.0	B
2	46.0	59544.576625	14.0	A
3	32.0	36399.189971	19.0	B
4	60.0	57140.638811	17.0	C

```
In [4]: # Iterative Imputer / Model-Based Imputation (Regresyon / tahmin)

# IterativeImputer her bir eksik değeri, diğer değişkenleri kullanarak ardışık
# tahminlerle doldurur
# – regresyon, bayesçi yöntem vs kullanılabilir. Bu, imputation sonrası değişkenle
# arası ilişkiyi korumaya çalışan güçlü bir yöntemdir.

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

df3 = df.copy()

imp_iter = IterativeImputer(random_state=42)
df3[num_cols] = imp_iter.fit_transform(df3[num_cols])

df3.head()
```

Out[4]:

	age	income	education_years	city
--	-----	--------	-----------------	------

0	56.0	21864.848417	17.0	C
1	69.0	29498.267917	10.0	B
2	46.0	59544.576625	14.0	A
3	32.0	36399.189971	19.0	B
4	60.0	57140.638811	17.0	C

```
In [5]: # Eksikliği "Flag / İşaretleme" + Imputation
# Eksik veri varlığı bilgisi önemli olabilir (örneğin "gelir yok" = farklı kateg

# Bu kod her sütun için "o hücre orijinalde eksik miydi?" bilgisini 0/1 ile sakl
# Daha sonra SimpleImputer / başka imputation yaparsınız – bu sayede model "orij
# bilgisini de görebilir. Bu flagging stratejisi, eksik verinin bilgi taşıdığı d

for col in df.columns:
    df[col + "_was_missing"] = df[col].isnull().astype(int)
df
```

Out[5]:

	age	income	education_years	city	age_was_missing	income_was_missing	ec
--	-----	--------	-----------------	------	-----------------	--------------------	----

0	56.0	21864.848417	17.0	C	0	0	
1	69.0	29498.267917	10.0	B	0	0	
2	46.0	59544.576625	14.0	A	0	0	
3	32.0	36399.189971	19.0	B	0	0	
4	60.0	57140.638811	17.0	C	0	0	
...	...	...	...	...	...	...	...
95	42.0	50984.737372	8.0	B	0	0	
96	62.0	67538.114704	11.0	B	0	0	
97	58.0	63099.754536	12.0	A	0	0	
98	46.0	53084.752010	12.0	C	0	0	
99	32.0	28347.749725	8.0	B	0	0	

100 rows × 8 columns



ffill / bfill — Ne demek, Ne Zaman Kullanılır?

- ffill (forward-fill): Eksik bir değer olduğunda, o sütunda bir önceki (üst satırdaki) geçerli değeri ileriye taşır. Yani "son bilinen değeri ileri taş" mantığı.
- bfill (backward-fill): Eksik bir değer olduğunda, o sütunda bir sonraki (alt satırdaki) geçerli değeri geriye taşır. Yani "sonraki bilinen değeri kullan" mantığı.

Bu yöntemler özellikle:

1. Zaman dizisi (time-series) verilerinde,
2. Ölçümlerin belli periyotlarla alındığı sensör / log / ardışık veri setlerinde
3. Sütunlarda ardışık eksik veriler varsa

gibi durumlarda mantıklı olur. Çünkü genelde “değer zamana göre çok değişmez / bir önceki değer yakın tahmin olabilir” ya da “sonraki değer, önceki bilinmeyene göre daha iyidir” gibi varsayımlar yapılabilir.

Ancak — dikkat: ffill / bfill kullanmadan önce veri bağlamını iyi değerlendirmek gerekir; yanlış kullanılırsa yanlış veri imputation’ı yapılmış olur.

```
In [6]: import pandas as pd
import numpy as np

# Örnek DataFrame: zamana / sıraya göre satırlar (örneğin sensör verisi ya da ar
df = pd.DataFrame({
    "time": pd.date_range(start="2025-01-01", periods=10, freq="D"),
    "temperature": [22.5, np.nan, np.nan, 23.0, 22.8, np.nan, 23.1, 23.3, np.nan,
    "humidity": [45, 46, np.nan, np.nan, 48, 49, np.nan, 50, 51, np.nan]
}))

print("Original veri:")
print(df)

# Forward fill – eksikleri önceki değerle doldur
df_ffill = df.ffmpeg()
print("\nForward-Fill edilmiş veri:")
print(df_ffill)

# Backward fill – eksikleri sonraki değerle doldur
df_bfill = df.bfill()
print("\nBackward-Fill edilmiş veri:")
print(df_bfill)
```

Original veri:

	time	temperature	humidity
0	2025-01-01	22.5	45.0
1	2025-01-02	NaN	46.0
2	2025-01-03	NaN	NaN
3	2025-01-04	23.0	NaN
4	2025-01-05	22.8	48.0
5	2025-01-06	NaN	49.0
6	2025-01-07	23.1	NaN
7	2025-01-08	23.3	50.0
8	2025-01-09	NaN	51.0
9	2025-01-10	23.5	NaN

Forward-Fill edilmiş veri:

	time	temperature	humidity
0	2025-01-01	22.5	45.0
1	2025-01-02	22.5	46.0
2	2025-01-03	22.5	46.0
3	2025-01-04	23.0	46.0
4	2025-01-05	22.8	48.0
5	2025-01-06	22.8	49.0
6	2025-01-07	23.1	49.0
7	2025-01-08	23.3	50.0
8	2025-01-09	23.3	51.0
9	2025-01-10	23.5	51.0

Backward-Fill edilmiş veri:

	time	temperature	humidity
0	2025-01-01	22.5	45.0
1	2025-01-02	23.0	46.0
2	2025-01-03	23.0	48.0
3	2025-01-04	23.0	48.0
4	2025-01-05	22.8	48.0
5	2025-01-06	23.1	49.0
6	2025-01-07	23.1	50.0
7	2025-01-08	23.3	50.0
8	2025-01-09	23.5	51.0
9	2025-01-10	23.5	NaN

In []:

In []:

In []:

In []:

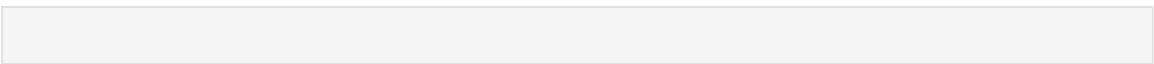
In []:

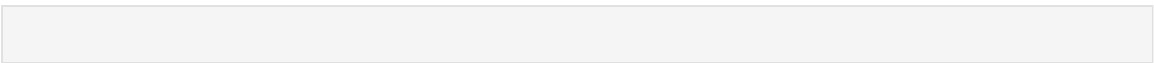
In []:

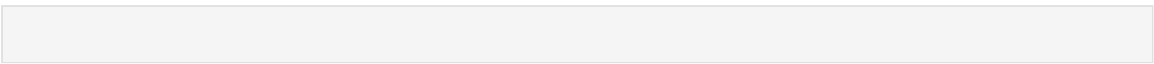
In []:

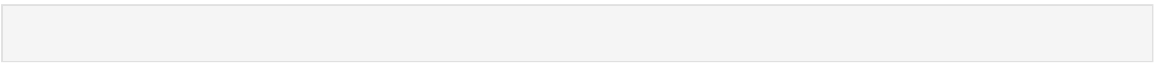
In []:

In []:

In []: 

In []: 

In []: 

In []: 

In []: 